Ontario Tech University
Faculty of Science, Computer Science — CSCI 3010U

Student Number: 100785833 April 10, 2026

2D Billiards Physics Simulation Using Python

by

Yash Patel

Abstract: For my CSCI 3010U course project, I built a two-dimensional billiards simulation using Python and the Pygame library. The simulation places eight balls on a table and models their motion, wall bouncing, and mutual collisions using real physics principles. Ball positions are updated with the Euler method for numerical integration, wall bouncing works by reversing the perpendicular velocity component, and ball–ball collisions are resolved using Newton's Law of Restitution with an impulse-based approach. A linear damping model simulates friction on the table surface. Three controlled experiments were conducted to study how the restitution coefficient and damping coefficient affect the total kinetic energy and total momentum of the system over time. All results matched theoretical predictions from physics, confirming that the simulation is physically valid.

Keywords: Billiards, Euler method, numerical integration, collision detection, Newton's Law of Restitution, damping, kinetic energy, conservation of momentum **Code Repository:** `https://github.com/yashp0529/billiards-simulation-project`

---

1 Introduction

Computer simulations allow us to study physical systems that would be difficult, expensive, or dangerous to test in real life. In my project, I chose to simulate a two-dimensional billiards table because billiards is a well-understood physical system where the results of a simulation can easily be verified by visual inspection and by comparing measurements against known physics formulas. I also chose billiards because the problem connects directly to the simulation techniques taught in CSCI 3010U: solving ordinary differential equations with a numerical integrator, detecting and resolving rigid-body collisions, and analysing energy and momentum as observable quantities that validate the simulation. The simulation I built handles four main tasks:

1. Updating each ball's position at every frame using the Euler method.
2. Detecting and responding to ball–wall collisions by reversing the appropriate velocity component.
3. Detecting and resolving ball–ball collisions using conservation of momentum and Newton's Law of Restitution.
4. Applying a linear damping model to simulate friction on the table surface.

Beyond building the simulation, I also ran three quantitative experiments to test whether the system behaves as physics theory predicts. This included tests for energy conservation in elastic collisions, exponential energy decay under friction, and momentum conservation during ball–ball collisions.

2 Related Work

Billiard ball simulation has a long history in both physics education and computer graphics. Gould, Tobochnik, and Christian [1], whose textbook is used in this course, present billiards as a standard benchmark for particle collision simulations and discuss the importance of using accurate numerical integration and correct collision response. Their work demonstrates that even a simplified 2D model can exhibit rich physical behaviour including chaotic trajectories and energy exchange between particles. In the computer graphics and game development literature, Ericson [2] describes the impulse-based collision response method in detail. This is the same method I used in my project: computing a collision normal, projecting relative velocities onto it, and applying an impulse to conserve momentum according to a chosen restitution coefficient. The approach is widely used in real-time game physics engines such as *Box2D* and *Bullet Physics* [3]. Earlier billiards simulations in academic settings (such as the "Billiard" project referenced in the CSCI 3010U course list of past projects) have generally focused on implementing the basic physics correctly. My project goes further by including a systematic experimental analysis of how restitution and damping parameters affect system energy and momentum, which adds a quantitative validation component that is less common in course-level implementations. The Pygame library [5] is used for rendering. While Pygame does not include a built-in physics engine, this is intentional: the goal of the project is to implement the physics from scratch using the mathematical models from lectures, rather than relying on an existing engine.

3 Methodology

Numerical Integration — the Euler Method

Each ball's motion is governed by ordinary differential equations. To simulate this on a computer, I discretise time with a step Δt and update positions using the **Euler method**:

$$x(t + \Delta t) = x(t) + v_x(t) \cdot \Delta t \tag{1}$$

$$y(t + \Delta t) = y(t) + v_y(t) \cdot \Delta t \tag{2}$$

The Euler method is a first-order integrator. I chose it because it is simple to implement and fast enough for a real-time simulation running at 120 frames per second. Its main drawback is that it introduces a small numerical error at each step: the predicted position drifts slightly from the true analytical solution. At $\Delta t = 1/120\,\mathrm{s}$, this error is negligible and does not affect the visible realism of the simulation. For scientific applications requiring higher precision, a fourth-order Runge–Kutta (RK4) method would be preferred.

Damping — Simulating Friction

I model friction by multiplying each velocity component by a decay factor at every time step:

$$v(t + \Delta t) = v(t) \cdot (1 - \lambda\, \Delta t) \tag{3}$$

where $\lambda \geq 0$ is the damping coefficient. This produces an exponential decay of speed over time, consistent with a linear drag force. Setting $\lambda = 0$ gives frictionless rolling; larger values bring balls to rest faster.

Ball–Wall Collisions

When a ball's edge crosses a table boundary, I reverse the velocity component perpendicular to that wall:

- Left or right wall: $v_x \leftarrow -v_x$
- Top or bottom wall: $v_y \leftarrow -v_y$

I also reposition the ball to remove any overlap with the boundary. This prevents *tunnelling* (a ball passing through the wall in a single time step at high speed). All wall collisions are perfectly elastic: no kinetic energy is lost.

Ball–Ball Collision Detection and Response

Two balls are colliding when:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \;<\; r_1 + r_2 \tag{4}$$

To resolve the collision, I compute a unit normal $\hat{n}$ from ball 1 toward ball 2, and the relative velocity along that normal:

$$dv_n = (\mathbf{v}_1 - \mathbf{v}_2) \cdot \hat{n} \tag{5}$$

If $dv_n \leq 0$ the balls are already separating and no action is needed. Otherwise the impulse magnitude is:

$$J = \frac{-(1+e) \cdot dv_n}{\dfrac{1}{m_1} + \dfrac{1}{m_2}} \tag{6}$$

where $e \in [0, 1]$ is the coefficient of restitution. The impulse acts along the collision normal $\hat{n}$ and redistributes velocity between the two balls — a faster ball slows down and the slower ball speeds up, exactly as in a real collision. The velocity updates are:

$$\mathbf{v}_1 \mathrel{+}= (J/m_1)\,\hat{n}, \qquad\qquad \mathbf{v}_2 \mathrel{-}= (J/m_2)\,\hat{n} \tag{7}$$

When $e = 1$ (elastic) kinetic energy is conserved. When $e < 1$ (inelastic) some energy is dissipated each collision.

Code Structure

The project consists of four Python files, all available at the GitHub repository listed at the top of this report:

- `ball.py` — `Ball` class with Euler update, wall collision, and kinetic energy methods.
- `physics.py` — `resolve_ball_collision()` function implementing Equation 6.
- `main.py` — Pygame window (1130×710 px, 120 FPS), live HUD with kinetic energy graph, and keyboard controls (↑↓ for damping, ←→ for restitution, `SPACE` to pause, `R` to reset).
- `analysis.py` — Headless experiment runner that records kinetic energy and momentum data and saves results as PNG graphs.

Figure 1 shows the simulation running.

4 Results

I ran three experiments using `analysis.py`, simulating 12 seconds of table time without a display window. All experiments use the same fixed initial conditions (random seed) so results are directly

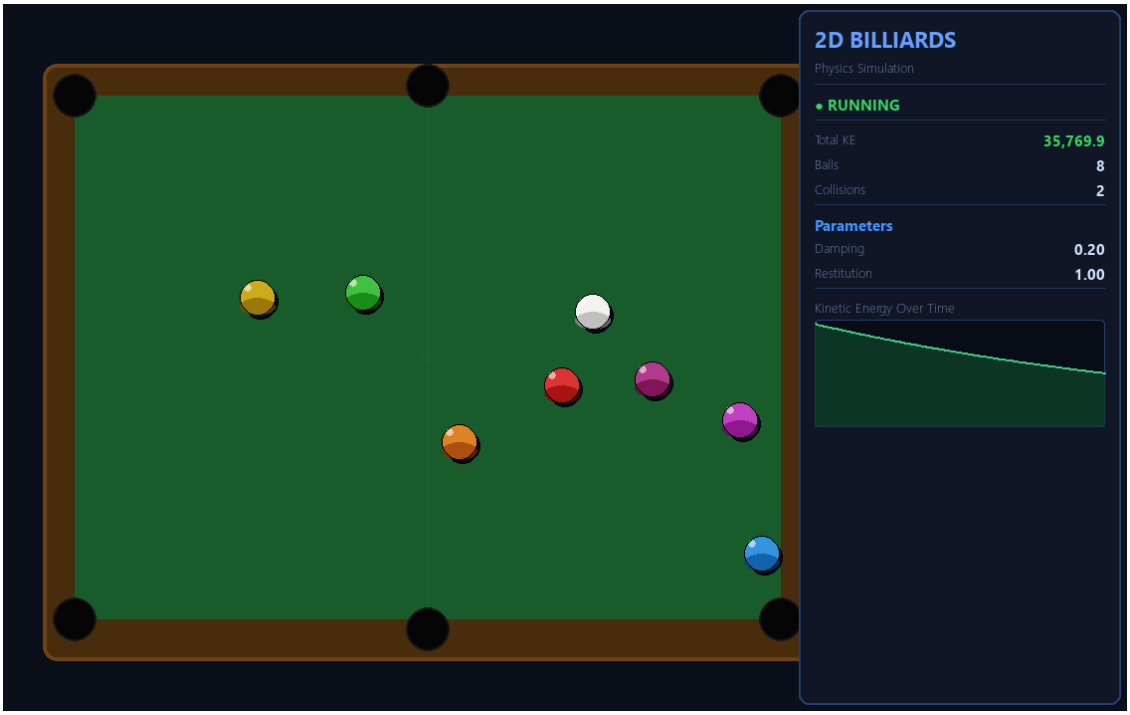

Figure 1: The interactive billiards simulation. Eight colour-coded balls move on a green felt table. The right panel shows the live kinetic energy graph, collision counter, elapsed time, and the current damping and restitution values.

comparable. Figure 2 shows all three plots.

Experiment 1 — Effect of Restitution on Kinetic Energy

Setup: $e \in \{1.0,\ 0.7,\ 0.4\}$, damping $\lambda = 0$. **Result:** At $e = 1.0$ the total kinetic energy is constant throughout – elastic collisions conserve KE exactly. At $e = 0.7$ and $e = 0.4$, the graph shows step-like decreases; each step corresponds to one ball–ball collision dissipating energy. Lower e produces larger drops per step. **Interpretation:** The simulation correctly implements Newton's Law of Restitution. The energy drop per collision grows as e decreases, which matches Equation 6.

Experiment 2 — Effect of Damping on Kinetic Energy

Setup: $\lambda \in \{0.0,\ 0.2,\ 0.6\}$, restitution $e = 1.0$. **Result:** At $\lambda = 0$ energy is constant. At $\lambda = 0.2$ energy decays smoothly along an exponential curve. At $\lambda = 0.6$ the decay is steep enough that balls stop within approximately four seconds. **Interpretation:** The smooth exponential decay

matches the analytical solution of the linear damping ODE, confirming that the friction model is correct.

Experiment 3 — Total Momentum Over Time

Setup: Elastic collisions, no damping; momentum magnitude tracked for 12 s. **Result:** The momentum magnitude shows sudden discrete jumps. Between jumps the value is stable. **Interpretation:** Each jump corresponds to a ball hitting a wall (an external force). Ball–ball collisions produce no jump because the impulse each ball receives is equal and opposite, so the total momentum is unchanged. This confirms Newton's Third Law and conservation of linear momentum within an isolated sub-system.

5 Discussion and Conclusions

Discussion

All three experiments produced results consistent with physics theory, which tells me the simulation is working correctly. One limitation worth noting is the **Euler method's numerical error**. Because the Euler method is first-order, it accumulates a small position error at each time step. I verified this is not a practical problem in my simulation by confirming that kinetic energy stays exactly flat under elastic, frictionless conditions (Experiment 1). If higher precision were required (for example, for scientific computation rather than real-time rendering), replacing Euler with RK4 would be the logical next step. The **brute-force collision detection** ($O(n^2)$ pairs checked every frame) is another limitation. For 8 balls this is 28 checks per frame and is fast enough. For a large number of balls a spatial partitioning structure (e.g., a uniform grid or quadtree) would be necessary.

Conclusions

This project gave me practical experience applying the core simulation techniques from CSCI 3010U to a physical system. I implemented:

- The Euler method for real-time numerical integration.
- Elastic wall collision response with positional correction.
- Impulse-based ball–ball collision using Newton's Law of Restitution.
- Linear damping to model surface friction.

The three quantitative experiments confirmed that these models are correct: elastic collisions conserved energy, lower restitution dissipated more energy per hit, higher damping produced

faster exponential decay, and momentum was conserved between ball–ball collisions. Future improvements I would consider include proper pocket detection, RK4 integration, and spatial partitioning for scalability. This project helped me see how the equations from lectures translate directly into working, physically accurate simulation code. **Code Repository:** `https://github.com/yashp0529/billiards-simulation-project`

References

[1] H. Gould, J. Tobochnik, and W. Christian, *An Introduction to Computer Simulation Methods: Applications to Physical Systems*, 3rd ed. Addison-Wesley, 2007.

[2] C. Ericson, *Real-Time Collision Detection*. Morgan Kaufmann, 2004.

[3] E. Coumans, *Bullet Physics Library*, 2024. `https://pybullet.org/`

[4] Course Notes, *CSCI 3010U – Simulation and Modeling*, Ontario Tech University, 2026. `http://csundergrad.science.uoit.ca/courses/sim-and-modeling-notes/`

[5] Pygame Community, *Pygame Documentation*, 2024. `https://www.pygame.org/docs/`

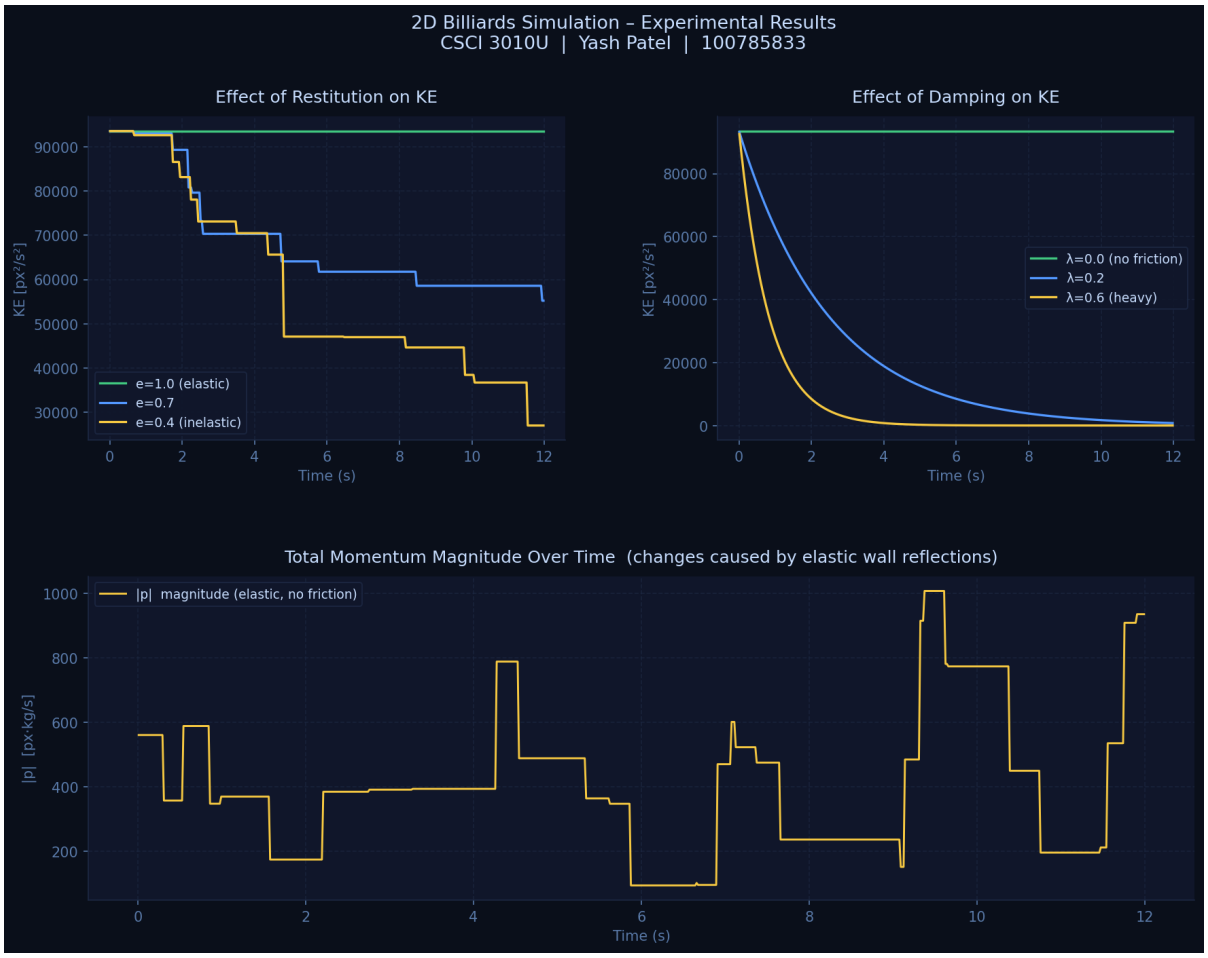

Figure 2: Experiment results. *Top-left:* KE vs. time for three restitution values. *Top-right:* KE vs. time for three damping values. *Bottom:* total momentum magnitude vs. time; jumps are caused by wall reflections.